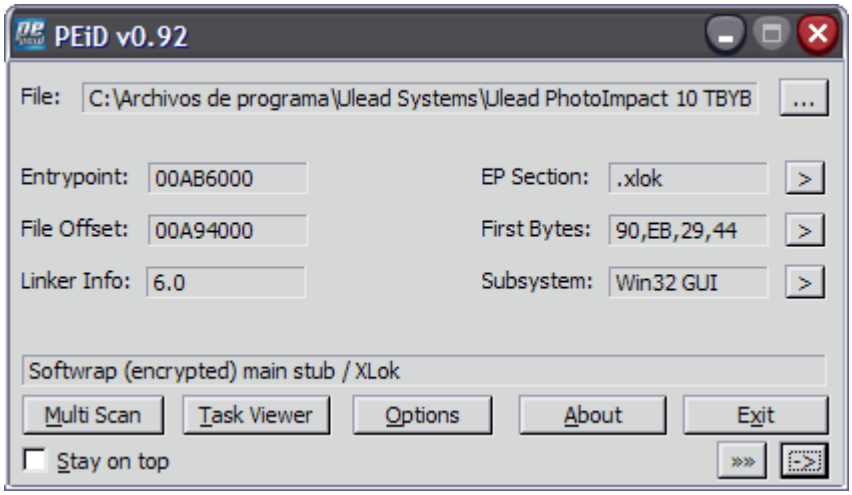
| Programa: | Ulead PhotoImpact 10 | |
|---|---|---|
| Protección: | Empaquetado con Softwrap y Trial 30 días | |
| Objetivos: | Desempaquetar el archivo (el Trial cae directo al desempakar, jeje..) | |
| URL descarga: | www.ulead.com | |
| Dificultad: | La verdad que no fue fácil | |
| Herramientas: | OllyDbg + PEiD + PEditor + ImpRec + HexWorkShop. | |
| Cracker: | SyXe'05 [hAcK-c0d3d] | Fecha: 05-08-2005 |

Hola a todo@s nuevamente..!!

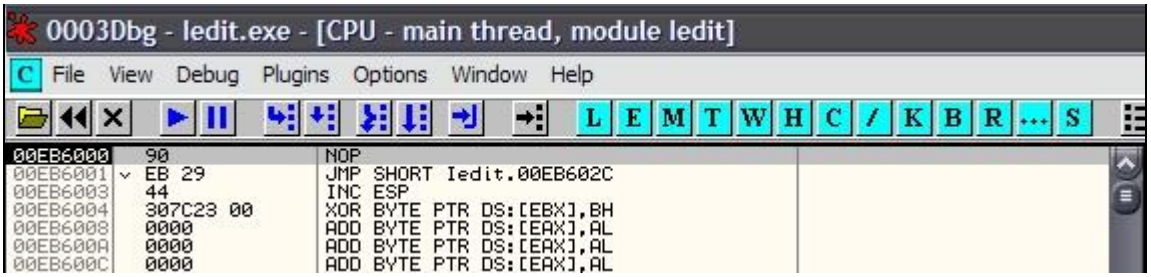
Pues estaba mirando entre los CD's típicos de las revistas ***Shareware*** y encontré este pequeñajo ;XDD se trata de ***Ulead PhotoImpact 10***, un potente editor de retoque fotográfico con el que podremos hacer grandes cosas, para los amantes de la fotografía, pero a nosotros esto no nos interesa, jeje.. nuestro objetivo está totalmente alejado y es el de desempaquetarlo. Primeramente lo instalamos y ya de momento vemos que nos ha metido mogollón de librerías y archivos además de varios ***.exes***, la aplicación en si está en Iedit.exe y desde aquí es desde donde lo arrancaremos, una vez instalado abrimos como siempre nuestro querido ***PEiD*** y dice ***Nothing found **** ...!!! cuando pasa esto a veces hay que picar en el botón de la flechita que hay en la parte inferior derecha y seleccionar ***Deep Scan*** o bien ***Hardcore Scan***, en este caso aparece esto:



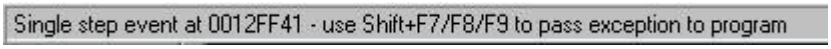

Ahora si nos dice el packer que es, la verdad es que yo no lo conocía, miré entre los tuts de ***CrackslatinoS*** y encontré uno en la ***Teoría 120*** resuelto magistralmente por ***Ricardo Narvaja*** pero su método no nos sirve ya que aquí el programa no es ninguna lanzadera y no se crea un segundo proceso sino que en el propio exe está todo, también encontré otro de ***Morales***, muy bien resuelto también pero tampoco sirvió de mucho porque era una aplicación en ***Visual Basic 5.0*** y no encontré más ¿es que no hay o es que no los vi? la cuestión es que encontré pocos textos de este compresor, también puede ser porque busqué muy poco y por eso me he decidido a escribir este tuto, espero que arroje un poco de luz sobre este Softwrap ***¿?***

Comencemos, como nunca nos hemos enfrentado a esta protección seremos cautos e iremos con tranquilidad recopilando el máximo de información posible, en una primera pasada tracearemos simplemente reconociendo el camino a recorrer después en una segunda pasada mucho más agresiva, jeje..

Cargamos Iedit.exe en ***OllyDbg*** y en el ***EP*** del packer empezamos a ver cosas raras ¡¡!!



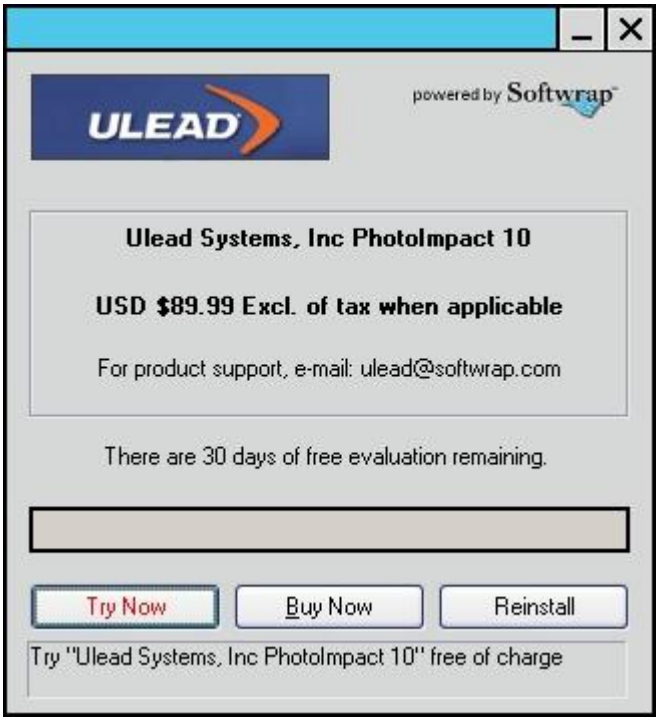
Que es esto! un ***NOP*** y no vemos ningún ***PUSHAD*** cercano, está claro que en este compresor han evitado de posibilidad de utilizar método del pushad para encontrar el ***OEP***, no nos desesperemos, iremos superando excepciones hasta la última y allí pondremos un ***BPM*** en la primera sección (el método de ***Ricardo Narvaja***), así que supongo que tenemos todos Olly configurado para que pare en las excepciones, no pongo imagen pues es muy sencillo: En ***Options -> Debug Options –> Exceptions*** destildamos todas las casillas excepto la primera que dice ***Ignore memory acces violations in KERNEL32***, con ***Alt+O*** aparece este cuadro también, tenemos ***Olly*** bien configurado? Si, pues damos RUN (***F9***) y comienza a detenerse, en la barra de estado tenemos qué tipo de excepción es en caso de habernos detenido por este motivo:



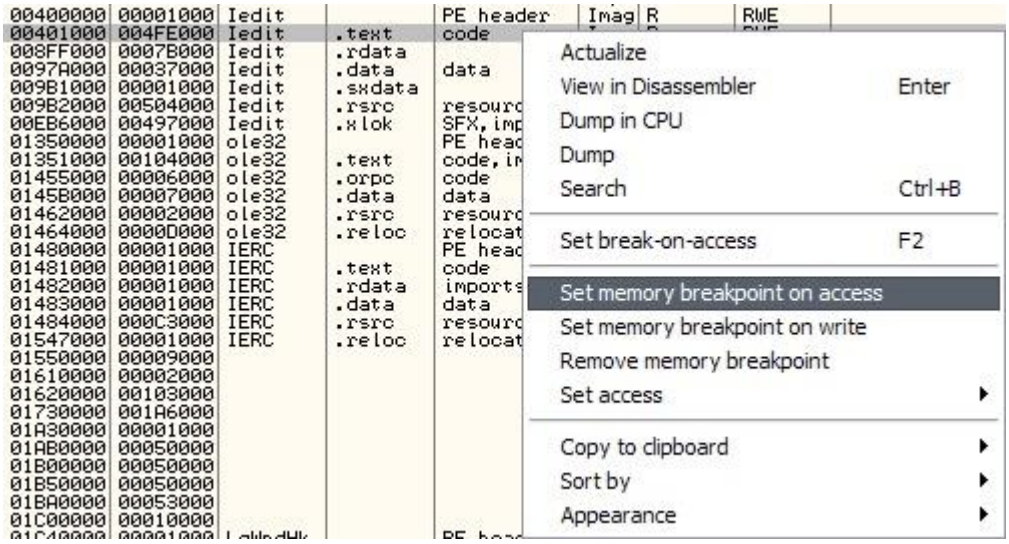

Vemos que este tipo de excepciones ocurren mucho y esto no nos deja tracear tranquilos, además las direcciones de memoria parecen mas a las de una pila que a otra cosa, son del tipo ***:12xxxx*** ¿? al llegar a esta excepción:

Si la superamos con ***Shift+F9*** aparece este otro mensaje:

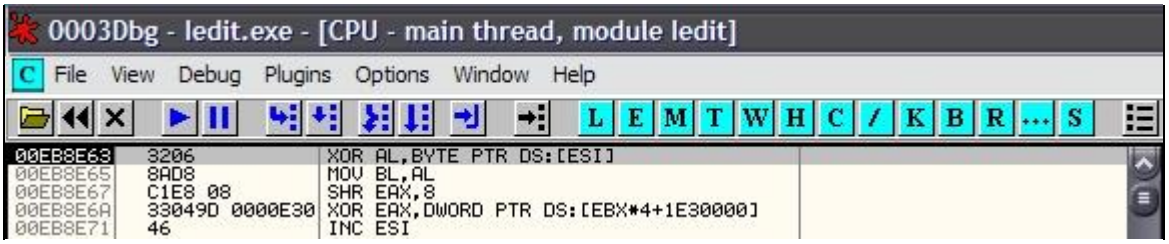
Nos informa de que el módulo ***softwrap*** tiene el ***Entry Point*** fuera de la sección ***.code***, lo que indica con casi total seguridad que está también empaquetado, se refiere lógicamente al archivo ***softwrap.dll*** sito en el mismo directorio de instalación del programa, acepamos y caemos en una nueva excepción. La pregunta es ¿El programa salta a la librería antes de acabar de desempaquetarse? lo lógico sería que primero se descomprimiera y a partir de aquí comience a cargar dll's pero como sabemos que esta dll no la carga el propio packer? la verdad no lo se, encontrar el ***OEP*** aquí no será tarea fácil, después del mensaje se suceden muchísimas excepciones, yo opté por deshabilitar algunas (ya sabéis como, en vez de tildar hay que destildar las casillas deseadas), al poco aparece la nag del programa en pantalla:




aprovecho para poner el ***BPM on Acces*** en la primera sección, recordad que a estos efectos la sección del ***PE header*** no la tenemos en cuenta, lo hacemos de esta manera:




Volvemos a la ventana de código con ***Alt+C***, aceptamos la nag pulsando en el botón ***Try Now*** y paramos aqui:




Miremos la barra de estado:

Mirad la importancia que tiene! nos está diciendo que paró por un ***Acceso de lectura*** y no de ***Ejecución***, o sea que no es lo que buscamos, nosotros, como es un loop quitaremos el BPM para que no se haga eterno y salimos poniendo un ***BPX*** en el final (antes del ret) y damos ***RUN***, esta operación se repite tres veces, cada vez quitamos el ***BPM*** y ***RUN*** y así, a la cuarta seguimos, vamos parando en excepciones la mar de raras ***¿?*** estamos cerca ya:

```
0003Dbg - ledit.exe - [CPU - main thread, module ledit]
File View Debug Plugins Options Window Help
00EB7068   5A          POP EDX
00EB7069   2A5A 8B     SUB BL,BYTE PTR DS:[EDX-75]
00EB706C   DB          ???                                  Unknown command
00EB706D   8BC0        MOV EAX,EAX
00EB706F   FFE0        JMP EAX
00EB7071   EB 04       JMP SHORT Iedit.00EB7077
00EB7073   DB          ???                                  Unknown command
00EB7074   24 0F       AND AL,0F
00EB7076   04 50       ADD AL,50
```

Ojo a este salto, miramos en la ventana de registros y vemos que ***EAX=8D01B0***, pulsamos ***F7*** y caemos allí:

```
0003Dbg - ledit.exe - [CPU - main thread, module ledit]
File View Debug Plugins Options Window Help
008D01B0   55          PUSH EBP
008D01B1   8BEC        MOV EBP,ESP
008D01B3   6A FF       PUSH -1
008D01B5   68 00079300 PUSH Iedit.00930700
008D01BA   68 1A038D00 PUSH Iedit.008D031A
008D01BF   64:A1 00000000 MOV EAX,DWORD PTR FS:[0]
008D01C5   50          PUSH EAX
008D01C6   64:8925 000000 MOV DWORD PTR FS:[0],ESP
008D01CD   83EC 68     SUB ESP,68
008D01D0   53          PUSH EBX
008D01D1   56          PUSH ESI
008D01D2   57          PUSH EDI
008D01D3   8965 E8     MOV DWORD PTR SS:[EBP-18],ESP
008D01D6   33DB        XOR EBX,EBX
008D01D8   895D FC     MOV DWORD PTR SS:[EBP-4],EBX
008D01DB   6A 02       PUSH 2
008D01DD   FF15 2C63EB00 CALL DWORD PTR DS:[EB632C]         Iedit.00EB8EA7
008D01E3   59          POP ECX
008D01E4   830D A00C9B00 OR DWORD PTR DS:[9B0CA0],FFFFFFFF
```

¡¡Este es nuestro OEP!!

Ya tenemos el primer dato importante, lo anotamos por supuesto, es el mismo que nos dio PEiD ;))

***<u>OEP</u>***: 8D01B0

Ahora busquemos la ***IAT***, en la misma imagen anterior se ve un salto a la misma en ***:8D01DD*** y vemos que apunta a ***:EB632C***, hacemos un ***Folow in Dump*** a ver que hay allí:

```
Address   Hex dump                                         ASCII
00EB632C  A7 8E EB 00 6B E9 AF 24 00 00 E9 C1 24 00 00 E9  §Ä╙.k╙»$..╙┴$..╙
00EB633C  06 26 00 00 E9 23 26 00 00 EB 00 01 60 EB 00 01  ♠&..╙#&..╙.`╙.`
00EB634C  60 EB 00 01 60 EB 00 01 60 EB 00 01 60 EB 00 01  `╙.`╙.`╙.`╙.`
00EB635C  60 EB 00 01 60 EB 00 01 60 EB 00 01 60 EB 00 01  `╙.`╙.`╙.`╙.`
00EB636C  60 EB 00 01 60 EB 00 01 60 EB 00 01 60 EB 00 01  `╙.`╙.`╙.`╙.`
00EB637C  60 EB 00 01 60 EB 00 01 60 EB 00 01 60 EB 00 01  `╙.`╙.`╙.`╙.`
00EB638C  60 EB 00 01 60 EB 00 01 60 EB 00 01 60 EB 00 01  `╙.`╙.`╙.`╙.`
00EB639C  60 EB 00 01 60 EB 00 01 60 EB 00 01 60 EB 00 01  `╙.`╙.`╙.`╙.`
```

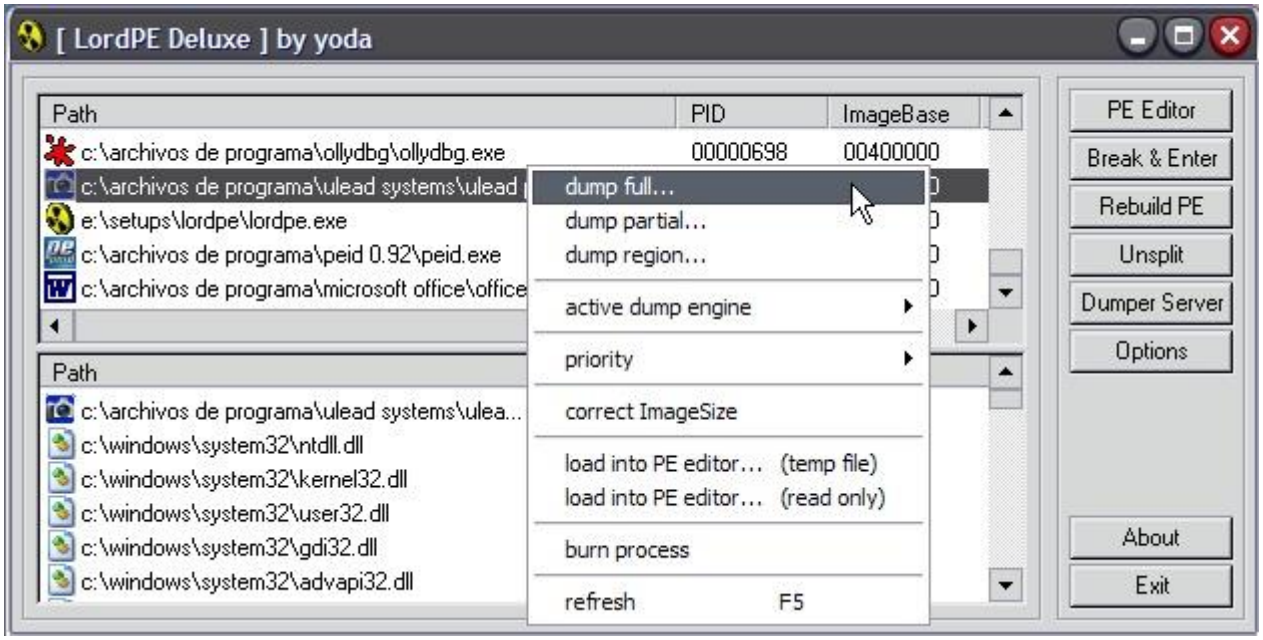
Vaya como está esta tabla..!! hay un único puntero digamos "algo decente", el resto son todos valores que apuntan hacia ***:0100EB60***, de aquí hasta ***:EB67A4*** no para de repetirse continuamente este valor y lo mismo hacia arriba hasta ***:EB6008***, esto no me gusta nada, mirad adonde apuntan todos los punteros:

```
0003Dbg - ledit.exe - [CPU - main thread, module ledit]
File View Debug Plugins Options Window Help
004014E5   FF15 2C63EB00   CALL DWORD PTR DS:[EB632C]          Iedit.00EB8EA7
004014EB   E8 414EAB00     CALL Iedit.00EB6331
004014F0   8D57 14         LEA EDX,DWORD PTR DS:[EDI+14]
004014F3   8B08            MOV ECX,DWORD PTR DS:[EAX]
004014F5   51              PUSH ECX
004014F6   53              PUSH EBX
004014F7   52              PUSH EDX
004014F8   8D4E 14         LEA ECX,DWORD PTR DS:[ESI+14]
004014FB   FF15 2C63EB00   CALL DWORD PTR DS:[EB632C]          Iedit.00EB8EA7
00401501   8B46 24         MOV EAX,DWORD PTR DS:[ESI+24]
00401504   3BC3            CMP EAX,EBX
00401506   74 0A           JE SHORT Iedit.00401512
00401508   50              PUSH EAX
00401509   FF15 2C63EB00   CALL DWORD PTR DS:[EB632C]          Iedit.00EB8EA7
0040150F   895E 24         MOV DWORD PTR DS:[ESI+24],EBX
00401512   395F 24         CMP DWORD PTR DS:[EDI+24],EBX
```

Apuntan todos hacia ***:EB632C***, luego hay otro grupo que apuntan a ***:EB6328*** como se ve en la imagen siguiente:

```
0003Dbg - ledit.exe - [CPU - main thread, module ledit]
File View Debug Plugins Options Window Help
008CCAA0   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAA6   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAAC   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAB2   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAB8   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCABE   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAC4   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCACA   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAD0   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAD6   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCADC   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAE2   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAE8   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAEE   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAF4   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCAFA   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
008CCB00   FF15 2863EB00   CALL DWORD PTR DS:[EB6328]          Iedit.00EB8E81
```

Definitivamente esta tabla está hecha pedazos, ya veremos como podemos reparar este desastre porque no tiene otro nombre, es más, yo nunca había visto semejante barbaridad ¡todos los punteros apuntando al mismo lugar! similar a lo que hace ***Vbox*** pero aún peor porque allí se llama a la misma ***dll*** que le da la ***api*** correcta pero esto se hace desde direcciones distintas de la ***iat***, pero es este caso sólo intervienen dos direcciones: ***[EB6328]*** y ***[EB63C]*** esto tiene muy mala reparación, lo digo en serio y nos vamos a encontrar con muchos problemas, de momento ahora que estamos en el ***OEP*** dumpearemos el archivo desde ***LordPE***, no olvidar activar la opción ***intellidump***:



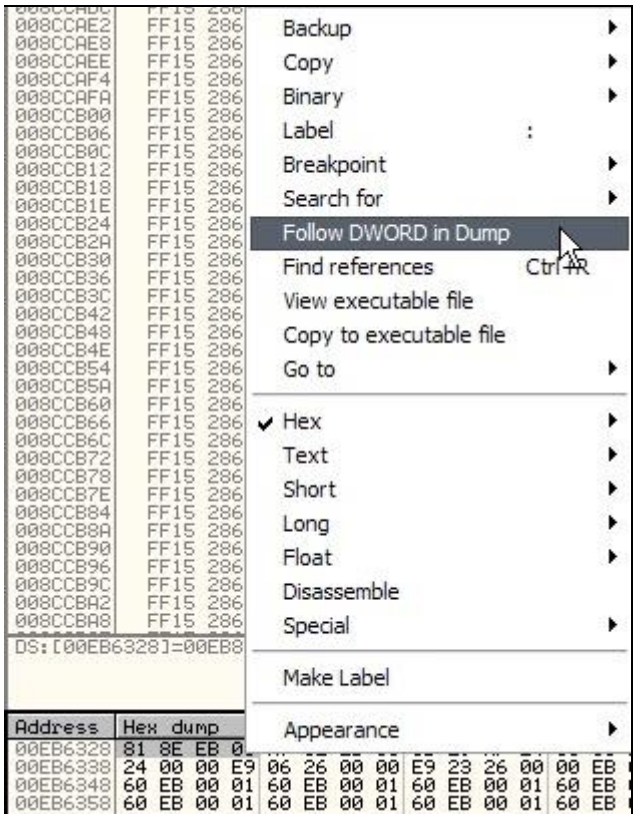
Le damos el nombre que nos pone por ejemplo que es ***dumped.exe***, asi nos referiremos seguro todos al mismo archivo, hecho esto tenemos el archivo volcado pero con una tabla de importaciones destrozada, vamos a intentar repararla, por donde comenzar ¿? buena pregunta...

Seguiremos a un puntero a ver donde nos lleva:

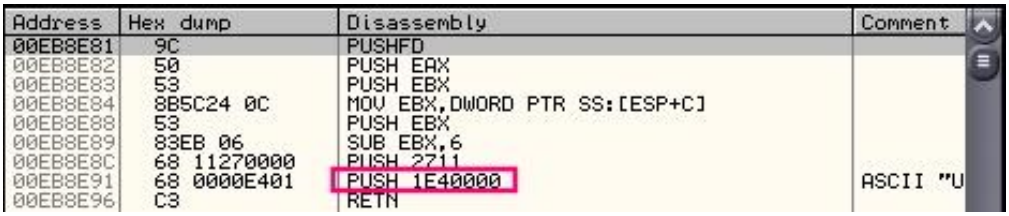
Lo haremos fácil, cogemos por ejemplo este puntero:

:008CCAA0 FF15 2863EB00 CALL DWORD PTR DS:[EB6328] ; Iedit.00EB8E81

hacemos un ***Follow in Dump***, alli seleccionamos los cuatro bytes de la entrada y ***Follow DWORD in Dump***, así vamos la dirección que marca, de esta manera lo haremos:




Alli en la ventana dump veremos los bytes que hay en la zona a la que apuntaba, con el botón derecho picamos y seleccionamos ***Disassemble*** para ver el código en ensamblador, asi se verá ahora:




Agarraos amigos que tenemos ***antidump***, ahi vemos un ***PUSH*** seguido de un ***RET*** que viene a ser como un salto a la zona marcada por la dirección que se ***pushea***, es similar hacer:

PUSH XX => Que => JMP XX
RET

o sea que aterrizamos en ***:1E40000*** tras superar el ***RET***, esto está muy bien pero en el archivo dumpeado esta dirección de memoria no existirá ya que la prepara el propio packer y recordad que lo hemos quitado del medio, de momento vamos al archivo original empaquetado y ponemos un ***BPX*** en esta parte a ver donde nos lleva ese retorno (***RET***):

```
0003Dbg - ledit.exe - [CPU - main thread]
File View Debug Plugins Options Window Help

01E40000   55              PUSH EBP
01E40001   56              PUSH ESI
01E40002   57              PUSH EDI
01E40003   51              PUSH ECX
01E40004   52              PUSH EDX
01E40005   50              PUSH EAX
01E40006   BD AE67EB00     MOV EBP,0EB67AE
01E4000B   016C24 18       ADD DWORD PTR SS:[ESP+18],EBP
01E4000F   2B9D 962A0000   SUB EBX,DWORD PTR SS:[EBP+2A96]
01E40015   8BBD DE2A0000   MOV EDI,DWORD PTR SS:[EBP+2ADE]
01E4001B   8B8D DA2A0000   MOV ECX,DWORD PTR SS:[EBP+2ADA]
01E40021   8BC3            MOV EAX,EBX
01E40023   2B85 D22A0000   SUB EAX,DWORD PTR SS:[EBP+2AD2]
01E40029   D3E8            SHR EAX,CL
01E4002B   8B0487          MOV EAX,DWORD PTR DS:[EDI+EAX*4]
01E4002E   8BF0            MOV ESI,EAX
01E40030   81E6 FFFF0000   AND ESI,0FFFF
01E40036   C1E8 10         SHR EAX,10
01E40039   8BF8            MOV EDI,EAX
01E4003B   8D043E          LEA EAX,DWORD PTR DS:[ESI+EDI]
01E4003E   BA A179DB33     MOV EDX,33DB79A1
01E40043   D1E8            SHR EAX,1
01E40045   8BC8            MOV ECX,EAX
01E40047   69C9 8466DA44   IMUL ECX,ECX,44DA6684
01E4004D   2BD1            SUB EDX,ECX
01E4004F   8B8D FA2A0000   MOV ECX,DWORD PTR SS:[EBP+2AFA]
01E40055   51              PUSH ECX
01E40056   8B0CC1          MOV ECX,DWORD PTR DS:[ECX+EAX*8]
01E40059   33CA            XOR ECX,EDX
01E4005B   3BCB            CMP ECX,EBX
01E4005D   59              POP ECX
01E4005E   74 0C           JE SHORT 01E4006C
01E40060   77 05           JA SHORT 01E40067
01E40062   8D78 01         LEA EDI,DWORD PTR DS:[EAX+1]
01E40065   EB D4           JMP SHORT 01E4003B
01E40067   8D70 FF         LEA ESI,DWORD PTR DS:[EAX-1]
01E4006A   EB CF           JMP SHORT 01E4003B
01E4006C   8B5C24 1C       MOV EBX,DWORD PTR SS:[ESP+1C]
01E40070   3354C1 04       XOR EDX,DWORD PTR DS:[ECX+EAX*8+4]
01E40074   0FB61B          MOVZX EBX,BYTE PTR DS:[EBX]
01E40077   2BD3            SUB EDX,EBX
01E40079   8BB5 EA2A0000   MOV ESI,DWORD PTR SS:[EBP+2AEA]
01E4007F   8B0496          MOV EAX,DWORD PTR DS:[ESI+EDX*4]
01E40082   35 D55A6CE0     XOR EAX,E06C5AD5
01E40087   50              PUSH EAX
01E40088   8A00            MOV AL,BYTE PTR DS:[EAX]
01E4008A   04 0F           ADD AL,0F
01E4008C   74 11           JE SHORT 01E4009F
01E4008E   F6D8            NEG AL
01E40090   2C 23           SUB AL,23
01E40092   74 0B           JE SHORT 01E4009F
01E40094   FEC8            DEC AL
```

Después de pasar por todas las operaciones que se le aplican ***EAX*** tiene en la dirección marcada la ***API*** correcta, esto nos da la posibilidad de injerto pero ¿donde vamos poniendo las apis? recordad que no tenemos un iat estándar, sino dos entradas que vienen precisamente a esta zona donde encuentran la api a ejecutar, esto es un gran inconveniente, es más hay ocasiones en que ***EAX*** no tiene ninguna dirección api sino otra tipo ***:01xxxxxx*** que nosotros tampoco tendremos en nuestro dumpeado y hay que tenerlo en cuenta, de hecho esta rutina que estamos viendo tampoco estará en nuestro ***dump***. Hay que pensar qué hacer, cómo atacar, por dónde empezar? como atajamos todo esto? sino hacemos nada el dumpeado nunca correrá ;( no vale ***PARA NADA*** y no nos gusta trabajar para nada, así que manos a la obra...

Lo primero que haremos será volcar las secciones que nos van a hacer falta en un posible injerto, veámoslas:

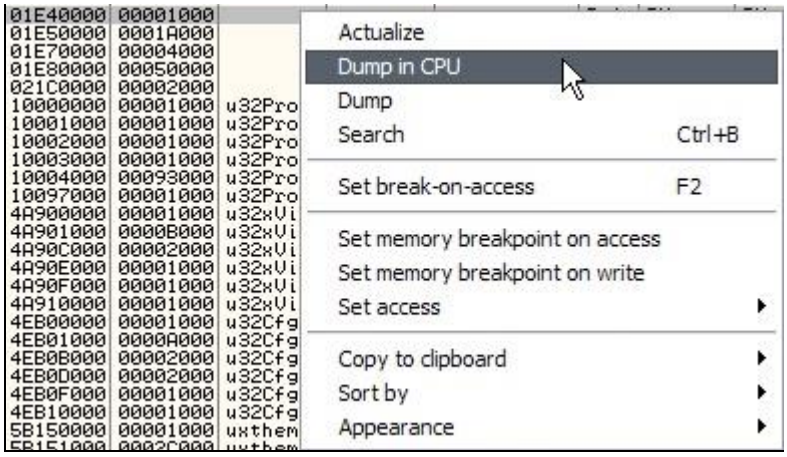
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 01E30000 | 00001000 | | | | Priv | RWE | RWE |
| 01E40000 | 00001000 | | | | Priv | RWE | RWE |
| 01E50000 | 0001A000 | | | | Priv | RWE | RWE |
| 01E70000 | 00004000 | | | | Priv | RWE | RWE |
| 01E80000 | 00050000 | | | | Priv | RWE | RWE |
| 01F80000 | 00001000 | | | | Priv | RW | RW |

Esto lo vemos en la ventana de memoria (***Alt+M***) y son las secciones del programa empaquetado y que en nuestro ***dump*** no están, de estas que vemos no todas nos hacen falta, puse un ***BPX*** en ***:01E40087*** e iba mirando que valores tomaba ***EAX*** para determinar las secciones que harían falta y no es nada alentador porque necesitamos 4 secciones de las ***6*** que vemos, son:

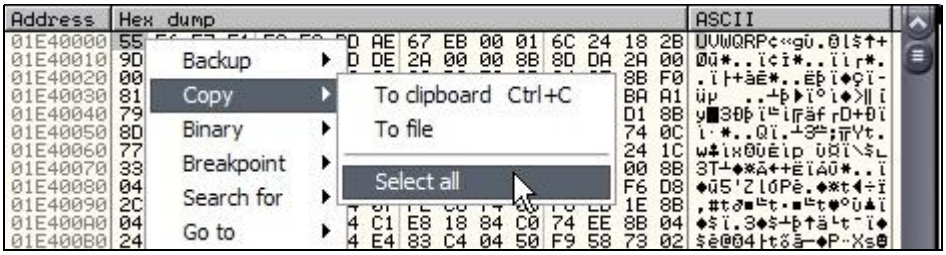
:01E40000, :01E50000, :01E70000 y :01E80000

estas tienen respectivamente un tamaño de 1000h, 1A000h, 4000h y 50000h..!! hay secciones de hasta 50000h bytes ¡vaya locura! de momento las vamos a volcar al disco a un archivo .bin, estos son los pasos:

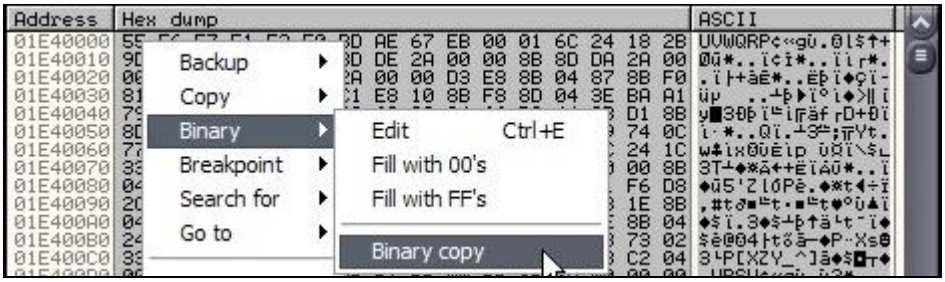
- Primero Volcamos primer a la ventana Dump:



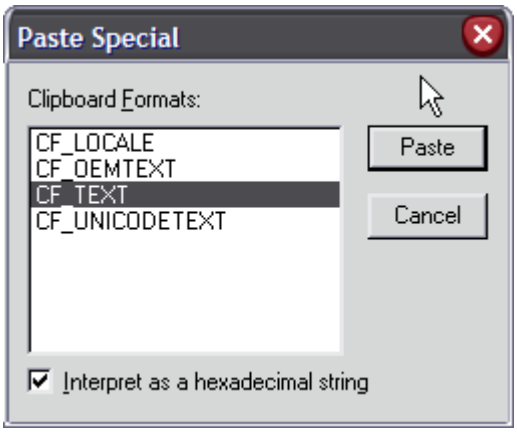

- Vamos a la ventana de Código (***Alt+C***) y seleccionamos los bytes de la ventana ***Dump***:




- Los copiamos con ***Binary Copy***:




Una vez copiados los bytes nos vamos al ***HexWorkShop***, abrimos un archivo nuevo, seleccionamos ***Edit -> Special...*** con el formato ***CF_TEXT*** como se ve en la imagen:




Le damos un nombre y listos, hacemos esto con las otras tres secciones restantes, ahora para referirnos todos a las mismas secciones le daremos el mismo nombre a cada una:

:01E40000 -> uno.bin -> 1000h bytes
:01E50000 -> cuatro.bin -> 1000h bytes
:01E70000 -> tres.bin -> 4000h bytes
:01E80000 -> dos.bin -> 50000h bytes

Disculpad por el orden pero el injerto lo fui construyendo sobre la marcha y me era muy trabajoso tener que estar cambiando constantemente los nombres (inicialmente pensé que son dos secciones sería suficiente).

Vale ya tenemos los archivos en el disco, pero antes de injertar tenemos que arreglar un poco dumped.exe ya que no tiene ***iat*** y nosotros necesitamos llamar tres ***apis*** en el injerto que son:

VirtualAlloc, CreateFileA y ReadFile.

Tenemos un problemón porque como sabemos sólo hay dos entradas reales de la tabla y que apuntan al mismo lugar, lo que sí que vemos es que hay espacio hacia arriba pero no es suficiente después veréis porqué os digo que falta espacio (menudo trabajito dio, bouuuff...) el problema está como digo en que tenemos que conseguir que importe las tres ***apis*** de arriba, la cosa no está fácil porque el programa de alguna manera calcula su dirección en la rutina desencriptadora que vimos antes, como consecuencia no hay ningún puntero que guarde la dirección real de la api sino que esta se obtiene mediante operaciones matemáticas, además si reparamos alguna de las dos entradas de la tabla y le ponemos la dirección por ejemplo de ***VirtualAlloc*** lo que estamos haciendo es dejar a todos los jmps y calls que vienen a esta entrada apuntando a esta api cuando en realidad necesita otra, y aunque no pasara esto tendríamos un archivo que sólo importa apis del ***kernel*** que es donde se ubica ***VirtualAlloc***.

Con este panorama se me ocurrió algo, y pensé ¡porque no intentarlo! y es digamos crear una nueva iat, anoté la dirección de cada una de las tres funciones que necesitamos en el injerto, desde el mismo ***Commandbar***:

Así con todas y obtuve (en mi máquina)

VirtualAlloc = 77e5ac72
CreateFileA = 77e5b476
ReadFile = 77e5ab4e

pero si ponemos estos valores en cualquier posición de memoria como por ejemplo:

[401210]= 77e5ac72 y luego hacemos:

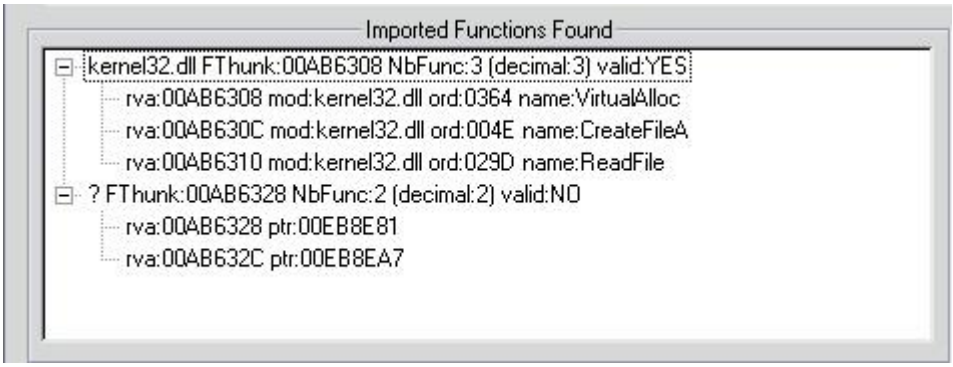
call dword ptr [401210]

parece que todo irá bien, y así será pero pruébalo en otra máquina ¡BOUUUUM! petó, esto es porque las direcciones de las apis cambian de una máquina a otra, hay que ser más profesionales ;XDDD necesitamos ayuda de ***Import Reconstructor*** para crear una nueva ***IT*** de manera que importe la librería ***kernel32.dll*** y entre ellas las funciones citadas y así podremos utilizarlo en cualquier ***PC*** ¿ok?

Por encima de los dos únicos punteros de la iat tenemos un sitiazo para utilizar, si vamos al ***Import*** y ponemos:

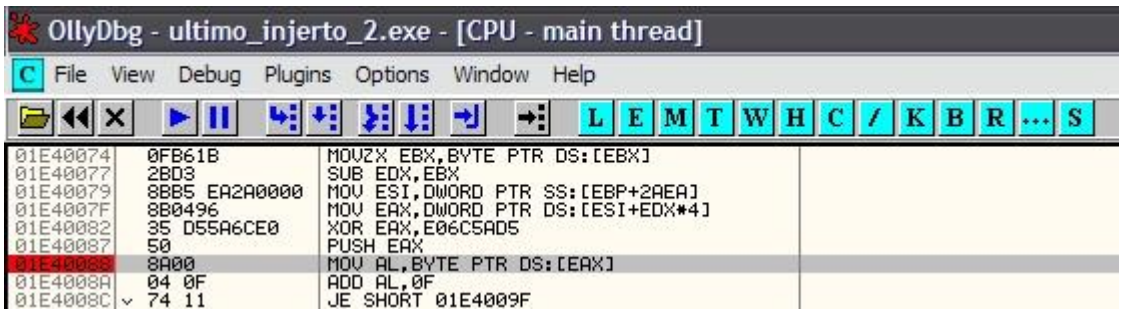
No cal decir pienso yo que hemos de seleccionar el proceso de Iedit.exe antes donde dice ***Attach to an Active Process***, después damos a ***Get Imports*** y nos saldrán todas las entradas inválidas, algo normal claro, pero

reparamos las tres apis que necesitamos, luego hacemos ***Cut Thunks*** mirando de no cargarnos las dos entradas adonde apuntan todos los jmps y calls del programa, quedaría así:




Luego podemos reparar el dumpeado simplemente pulsando ***Fix Dump*** y seleccionándolo, ahora tenemos el archivo que importa las tres apis que necesitamos sin molestar el funcionamiento de los otros dos punteros ;)

Pero no nos alarmemos todavía, ejecutamos el ***dumped_.exe*** y ponemos un ***BPX*** al inicio de la rutina, antes del ***PUSH EAX*** donde ***EAX*** toma el valor api correcto vemos que se produce una excepción que no nos deja depurar más el programa, al principio no sabía que pasaba pero fui al proceso original empaquetado, puse un ***BPX*** allí y me fijé en la diferencia:




Esta es la zona conflictiva, miremos los registros en el original y en el dumpeado:

***ORIGINAL***:

***DUMPEADO***:

(la foto se tomó desde dos posiciones distintas pero esto no importa)

La primera imagen es del original y la segunda (derecha) la del dumpeado, como vemos llegan al lugar conflictivo con el mismo valor pero en el original lo reconoce como ***MSVCRT.__set_app_type*** y en el dumpeado no lo ve, para él es un simple valor más, esto ocurre porque no tenemos la librería ***MSVCRT.dll*** entre las funciones importadas lo que es un grave problema si tenemos en cuenta que este programita importa mas de ***150*** librerías..!! La cena está servida muchachos, tenemos mucho pero que muuuuucho trabajo aquí, debemos conseguir que nuestro dumpeado importe todas las ***dll's*** que se necesitan en el proceso original ¿y cómo se hace esto? yo hasta ahora cada vez que he necesitado añadir un Descriptor (se dice así, mirarlo en el ***LordPE*** ya lo veréis) iba precisamente al citado ***LordPE*** y lo añadía, esta función también la incorpora ***PEditor*** pero no me fio mucho, yo había pensado en hacerlo de otra manera más manual que no quiere decir que sea mejor, lo que yo hice fue buscar una zona grande para albergar una falsa iat donde irán los punteros de las dll's que necesitemos, por esto os decía antes que la otra iat era pequeña ya que allí no cabrían todas, esto además no va a interferir en la otra tabla ya que no le molesta, es digamos que un refuerzo, su única misión es la de conseguir importar las librerías necesarias ya lo hemos dicho y ya de paso importar ***VirtualAlloc***, ***CreateFileA*** y ***ReadFile***, jeje.. como lo vamos a hacer desde

ImpRec funcionará en cualquier máquina, luego la verdadera iat la solventaremos mediante un injerto ¿os parece? pues vamos a la carga...

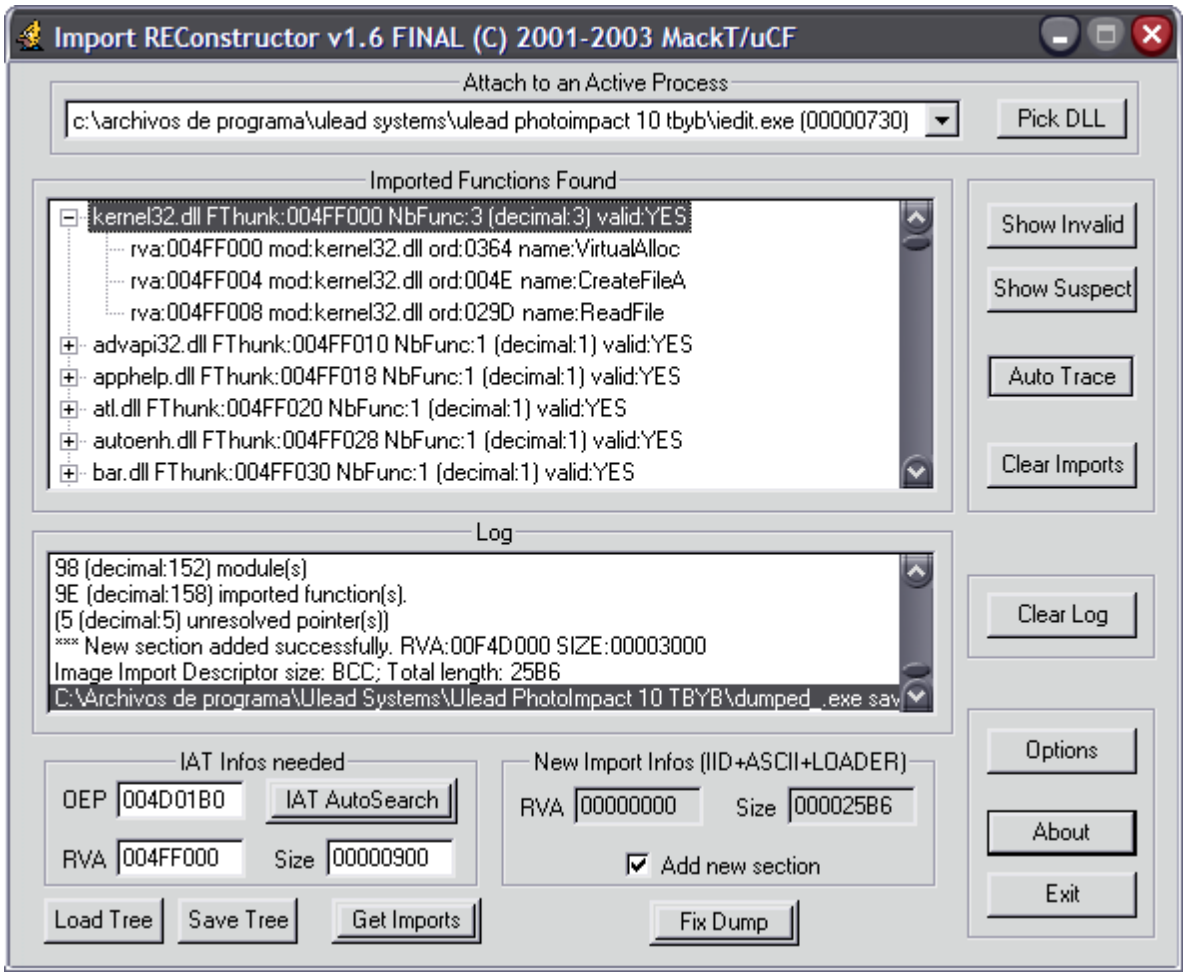
Lo primero claro es buscar ese espacio libre, tenemos uno bien bonito a partir de ***:8FF000*** que es el inicio de la seccion ***.rdata*** que presumiblemente podría albergar la verdadera ***iat***, veamos:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 00400000 | 00001000 | Iedit_cr | | PE header | Imag | R | RWE |
| 00401000 | 004FE000 | Iedit_cr | .text | code | Imag | R | RWE |
| 008FF000 | 0007B000 | Iedit_cr | .rdata | | Imag | R | RWE |
| 0097A000 | 00037000 | Iedit_cr | .data | data | Imag | R | RWE |
| 009B1000 | 00001000 | Iedit_cr | .sxdata | | Imag | R | RWE |

Vamos a ver que hay alli, picamos con el botón derecho y elegimos ***Dump in CPU***, luego vamos y vemos esto:

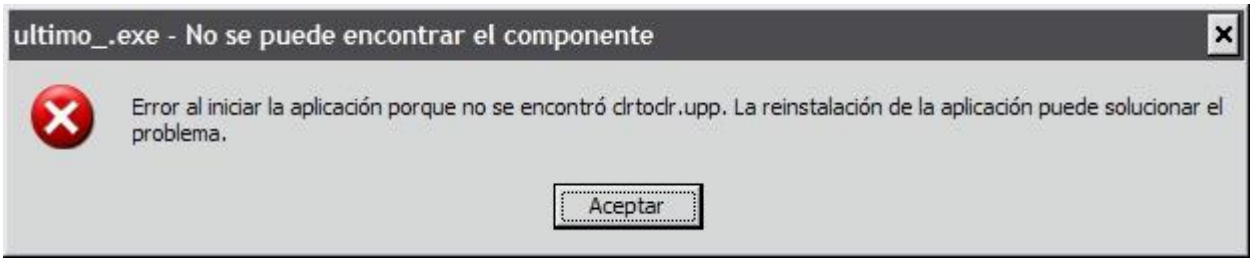
```
Address   Hex dump                                          ASCII
008FF000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF090  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Pues está vacía como se puede ver, jeje.. tiene todos los puntos para ser nuestro lugar, aquí montaremos nuestra propia tabla para reparar el error anterior ya que así conseguiremos importar las ***dll's***, luego de las funciones no nos deberemos preocupar, pues sólo es necesaria la base de la dll y a partir de aquí se obtiene la dirección de la api a utilizar, por eso esto funcionará seguramente. El objetivo es ir al ***Import Reconstructor*** y allí ir reparando los punteros que verá como inválidos de esta zona, antes rellenaremos todo con unos valores cualquiera sino los ignorará y no mostrará nada, luego repararemos las tres primeras para ***VirtualAlloc***, ***CreateFileA*** y ***ReadFile*** respectivamente, dejaremos un puntero de separación y reparamos el siguiente pero para otra ***dll***, así con todas las que necesitemos dejando sobretodo un espacio entre ellas no lo olvidéis cuando estén todas picamos en ***Show Invalid*** y luego en ***Cut Thunks***, el ***ImpRec*** tendrá este aspecto: (adjunto archivo falsa_IAT.txt)



Dejé los tres punteros del ***Kernel*** a la vista para que se entienda lo que quería explicar, pues no se me hace nada fácil explicar algunas cosas así con palabras, vemos además muchas otras librerías, jeje.. justo lo que necesitamos le damos a ***Fix Dump*** y tenemos reparado parte del asunto pero no del todo luego lo acabamos.

NOTA: Si te aparece este mensaje al cargar el exe en Olly:




quiere decir que le hemos añadido una librería que no necesita, la eliminamos con ***Cut Thunk***, tiramos el archivo reparado y le hacemos ***Fix Dump*** nuevamente al dumpeado, así hasta que no se queje.

Vamos a por el tema del injerto, necesitamos cargar en memoria ***4*** bloques llamémoslo así cada uno de un tamaño distinto y además en una zona concreta de la memoria, no nos vale con que nos los cargue en ***:02000000*** sino que tiene que ser en el lugar que nosotros indiquemos, pero antes necesitamos saber unas cuantas cosas de las funciones que vamos a utilizar, un poco de teoría:

**VirtualAlloc**:

```
LPVOID VirtualAlloc(
   LPVOID lpAddress,          // dirección de la zona de memoria a crear/reservar
   DWORD dwSize,              // tamaño en bytes
   DWORD flAllocationType,    //Tipo de solicitud
   DWORD flProtect            //Tipo de acceso
);
```

Esta función básicamente para lo que la utilizaremos será para reservar un espacio de memoria o bien para pedirlo directamente, esto se lo haremos saber con el parámetro ***flAllocationType***, un ***2000*** indicará reserva y un ***3000*** que la cree directamente, antes se le pasa el tipo de acceso a la zona, lectura, escritura, ambas, ninguna, etc., después el tamaño de la zona a crear/reservar y por último la dirección donde queremos que la reserve o cree, este último parámetro si es cero (***NULL***) el sistema decidirá la dirección.

**CrateFileA**:

```
HANDLE CreateFile(
   LPCTSTR lpFileName,                        // puntero al nombre del archivo
   DWORD dwDesiredAccess,                     // modo de acceso (lectura-escritura)
   DWORD dwShareMode,                         // modo compartido
   LPSECURITY_ATTRIBUTES lpSecurityAttributes, // punteo a atributos de seguridad
   DWORD dwCreationDistribution,              // como crear archivo
   DWORD dwFlagsAndAttributes,                // atributos del archivo
   HANDLE hTemplateFile                       // manejador del archivo con atributos de copia
);
```

Esta viene en inglés pero más o menos lo que importa aqui es que esta función sirve tanto para crear un archivo como para digamos abrirlo, en cualquier caso devuelve el manejador (***handle***) del archivo en ***EAX*** si la operación tuvo éxito o bien ***EAX=ffffffff*** (-1) si fue fallida. A nosotros nos interesa el manejador y luego pasárselo a la función ReadFile para que pueda acceder a nuestro archivo .bin.

**ReadFile**:

```
BOOL ReadFile(
   HANDLE hFile,                              // handle of file to read
```

```
   LPVOID lpBuffer,                    // dirección del buffer de destino
   DWORD nNumberOfBytesToRead,         // número de bytes a leer
   LPDWORD lpNumberOfBytesRead,        // dirección del número de bytes a leer
   LPOVERLAPPED lpOverlapped           // dirección de la estructura de datos
);
```

Esta función se suele utilizar para leer el contenido de un archivo y almacenarlo en la dirección deseada, esto se le dice mediante el parámetro ***lpBuffer***, luego ***hFile*** significa el manejador del archivo a abrir que habremos obtenido previamente mediante una llamada a ***CreateFileA***, el número de bytes a leer se le ha de decir también (parámetro ***nNumberOfBytesToRead***), y del resto pasamos.

Ahora sí estamos semi-preparados (jeje..) para crear el injerto, pero esto no es todo, necesitamos situar en algún lugar de la memoria el nombre de los ***4*** archivos y en otro sus respectivos tamaños, yo los puse aquí:

VirtualAlloc en [8FF000]
CreateFileA en [8FF004]
ReadFile en [8FF008]

necesitaremos algunas variables sonde almacenar el nombre del archivo, el tamaño de la sección a crear. Estas serán por ejemplo las siguientes:

```
cuatro.bin  -> [f08fe0]  -> su tamaño 1A000h en  [f08fec]
   tres.bin  -> [f08ff0]  -> su tamaño    4000h en  [f08ffc]
   uno.bin  -> [f09000] -> su tamaño    1000h en  [f0900c]
   dos.bin  -> [f09010] -> su tamaño  50000h en  [f0901c]
```

Ahora sí podemos construir el injerto, una vez más pedir disculpas si no está todo tan ordenado como a mi me gustaría, pensar que esto me costó sacarlo y fueron muchos intentos y pruebas, ir reajustando todo en cada prueba me hubiera llevado muchísimo más tiempo ;))

```
------------------------/ Injerto /------------------------
00F09020   60              PUSHAD                          ; Salvamos los registros en la pila
00F09021   6A 04           PUSH 4                          ; Código ejecutable y lectura-escritura
00F09023   68 00300000     PUSH 3000                       ; le pedimos que cree ya la zona
00F09028   68 00100000     PUSH 1000                       ; tamaño de esta
00F0902D   68 0000E401     PUSH 1E40000                    ; y su dirección
00F09032   FF15 00F08F00   CALL DWORD PTR DS:[8FF000]      ; kernel32.VirtualAlloc
00F09038   85C0            TEST EAX,EAX
00F0903A   6A 04           PUSH 4                          ; Código ejecutable y lectura-escritura
00F0903C   68 00300000     PUSH 3000                       ; le pedimos que cree ya la zona
00F09041   68 00000500     PUSH 50000                      ; de tamaño 50000h bytes
00F09046   68 0000E801     PUSH 1E80000                    ; a partir de la dirección de memoria 1E80000
00F0904B   FF15 00F08F00   CALL DWORD PTR DS:[8FF000]      ; kernel32.VirtualAlloc
00F09051   85C0            TEST EAX,EAX
00F09053   6A 04           PUSH 4                          ; Código ejecutable y lectura-escritura
00F09055   68 00300000     PUSH 3000                       ; le pedimos que cree ya la zona
00F0905A   68 00400000     PUSH 4000                       ; de tamaño 4000h bytes
00F0905F   68 0000E701     PUSH 1E70000                    ; a partir de la dirección de memoria 1E70000
00F09064   FF15 00F08F00   CALL DWORD PTR DS:[8FF000]      ; kernel32.VirtualAlloc
00F0906A   85C0            TEST EAX,EAX
00F0906C   6A 00           PUSH 0                          ; primer argumento de la función
00F0906E   6A 00           PUSH 0                          ; . . .
00F09070   6A 03           PUSH 3                          ; . . .
00F09072   6A 00           PUSH 0                          ; . . .
00F09074   6A 00           PUSH 0                          ; . . .
00F09076   68 00000080     PUSH 80000000                   ; . . .
00F0907B   68 0090F000     PUSH dumped_.00F09000           ; ASCII "uno.bin"  /*puntero al nombre*/
00F09080   FF15 04F08F00   CALL DWORD PTR DS:[8FF004]      ; kernel32.CreateFileA
00F09086   85C0            TEST EAX,EAX
00F09088   6A 00           PUSH 0                          ; primer argumento
00F0908A   68 0C90F000     PUSH dumped_00F0900C            ; puntero a la longitud del archivo a leer
00F0908F   68 00100000     PUSH 1000                       ; tamaño de bytes a leer (en hexa)
```

```
00F09094   68 0000E401        PUSH 1E40000                  ; buffer de almacenamiento
00F09099   50                 PUSH EAX                      ; manejador del archivo a leer
00F0909A   FF15 08F08F00      CALL DWORD PTR DS:[8FF008]    ; kernel32.ReadFile
00F090A0   85C0               TEST EAX,EAX
00F090A2   6A 00              PUSH 0                        ; primer argumento de la función
00F090A4   6A 00              PUSH 0                        ; . . .
00F090A6   6A 03              PUSH 3                        ; . . .
00F090A8   6A 00              PUSH 0                        ; . . .
00F090AA   6A 00              PUSH 0                        ; . . .
00F090AC   68 00000080        PUSH 80000000                 ; . . .
00F090B1   68 1090F000        PUSH dumped_.00F09010         ; ASCII "dos.bin"
00F090B6   FF15 04F08F00      CALL DWORD PTR DS:[8FF004]    ; kernel32.CreateFileA
00F090BC   85C0               TEST EAX,EAX
00F090BE   6A 00              PUSH 0                        ; primer argumento
00F090C0   68 1C90F000        PUSH dumped_.00F0901C         ; puntero a la longitud del archivo a leer
00F090C5   68 00000500        PUSH 50000                    ; tamaño de bytes a leer (en hexa)
00F090CA   68 0000E801        PUSH 1E80000                  ; buffer de almacenamiento
00F090CF   50                 PUSH EAX                      ; manejador del archivo a leer
00F090D0   FF15 08F08F00      CALL DWORD PTR DS:[8FF008]    ; kernel32.ReadFile
00F090D6   85C0               TEST EAX,EAX
00F090D8   6A 00              PUSH 0                        ; primer argumento de la función
00F090DA   6A 00              PUSH 0                        ; . . .
00F090DC   6A 03              PUSH 3                        ; . . .
00F090DE   6A 00              PUSH 0                        ; . . .
00F090E0   6A 00              PUSH 0                        ; . . .
00F090E2   68 00000080        PUSH 80000000                 ; . . .
00F090E7   68 F08FF000        PUSH dumped_.00F08FF0         ; ASCII "tres.bin"
00F090EC   FF15 04F08F00      CALL DWORD PTR DS:[8FF004]    ; kernel32.CreateFileA
00F090F2   85C0               TEST EAX,EAX
00F090F4   6A 00              PUSH 0                        ; primer argumento
00F090F6   68 FC8FF000        PUSH dumped_.00F08FFC         ; puntero a la longitud del archivo a leer
00F090FB   68 00400000        PUSH 4000                     ; tamaño de bytes a leer (en hexa)
00F09100   68 0000E701        PUSH 1E70000                  ; buffer de almacenamiento
00F09105   50                 PUSH EAX                      ; manejador del archivo a leer
00F09106   FF15 08F08F00      CALL DWORD PTR DS:[8FF008]    ; kernel32.ReadFile
00F0910C   85C0               TEST EAX,EAX
00F0910E   6A 04              PUSH 4                        ; Código ejecutable y lectura-escritura
00F09110   68 00300000        PUSH 3000                     ; le pedimos que cree ya la zona
00F09115   68 00A00100        PUSH 1A000                    ; y su dirección
00F0911A   68 0000E501        PUSH 1E50000                  ; y su dirección
00F0911F   FF15 00F08F00      CALL DWORD PTR DS:[8FF000]    ; kernel32.VirtualAlloc
00F09125   85C0               TEST EAX,EAX
00F09127   6A 00              PUSH 0                        ; primer argumento de la función
00F09129   6A 00              PUSH 0                        ; . . .
00F0912B   6A 03              PUSH 3                        ; . . .
00F0912D   6A 00              PUSH 0                        ; . . .
00F0912F   6A 00              PUSH 0                        ; . . .
00F09131   68 00000080        PUSH 80000000                 ; . . .
00F09136   68 E08FF000        PUSH dumped_.00F08FE0         ; ASCII "cuatro.bin"
00F0913B   FF15 04F08F00      CALL DWORD PTR DS:[8FF004]    ; kernel32.CreateFileA
00F09141   85C0               TEST EAX,EAX
00F09143   6A 00              PUSH 0                        ; primer argumento
00F09145   68 EC8FF000        PUSH dumped_.00F08FEC         ; puntero a la longitud del archivo a leer
00F0914A   68 00A00100        PUSH 1A000                    ; tamaño de bytes a leer (en hexa)
00F0914F   68 0000E501        PUSH 1E50000                  ; buffer de almacenamiento
00F09154   50                 PUSH EAX                      ; manejador del archivo a leer
00F09155   FF15 08F08F00      CALL DWORD PTR DS:[8FF008]    ; kernel32.ReadFile
00F0915B   85C0               TEST EAX,EAX
00F0915D   61                 POPAD                         ; Restauramos los registros
00F0915E   E9 4D709CFF        JMP dumped_.008D01B0          ; y saltamos al OEP..!!
00F09163   90                 NOP
```

----------/ Injerto /----------

Los bytes en binario del injerto los pongo aquí por si acaso:

60 6A 04 68 00 30 00 00 68 00 10 00 00 68 00 00 E4 01 FF 15 00 F0 8F 00 85 C0 6A 04 68 00 30 00 00 68 00 00 05 00 68 00 00 E8 01 FF 15 00 F0 8F 00 85 C0 6A 04 68 00 30 00 00 68 00 40 00 00 68 00 00 E7 01 FF 15 00 F0 8F 00 85 C0 6A 00 6A 00 6A 03 6A 00 6A 00 68 00 00 00 80 68 00 90 F0 00 FF 15 04 F0 8F 00 85 C0 6A 00 68 0C 90 F0 00 68 00 10 00 00 68 00 00 E4 01 50 FF 15 08 F0 8F 00 85 C0 6A 00 6A 00 6A 03 6A 00 6A 00 68 00 00 00 80 68 10 90 F0 00 FF 15 04 F0 8F 00 85 C0 6A 00 68 1C 90 F0 00 68 00 00 05 00 68 00 00 E8 01 50 FF 15 08 F0 8F 00 85 C0 6A 00 6A 00 6A 03 6A 00 6A 00 68 00 00 00 80 68 F0 8F F0 00 FF 15 04 F0 8F 00 85 C0 6A 00 68 FC 8F F0 00 68 00 40 00 00 68 00 00 E7 01 50 FF 15 08 F0 8F 00 85 C0 6A 04 68 00 30 00 00 68 00 A0 01 00 68 00 00 E5 01 FF 15 00 F0 8F 00 85 C0 6A 00 6A 00 6A 03 6A 00 6A 00 68 00 00 00 80 68 E0 8F F0 00 FF 15 04 F0 8F 00 85 C0 6A 00 68 EC 8F F0 00 68 00 A0 01 00 68 00 00 E5 01 50 FF 15 08 F0 8F 00 85 C0 61 E9 4D 70 9C FF 90

Bueno espero que aún estéis ahí, jeje.. es broma hombre, esto básicamente lo que hace es encargarse de crear los 4 espacios de memoria, acceder a los archivos que hemos dumpeado y que tenemos en formato ***.bin*** coger su contenido y pasarlo a la memoria, ya está, acordaos de guardar el nombre de los archivos y su tamaño:

```
Address   Hex dump                                          ASCII
00F08FE0  63 75 61 74 72 6F 2E 62 69 6E 00 00 00 A0 01 00  cuatro.bin...á0.
00F08FF0  74 72 65 73 2E 62 69 6E 00 00 00 00 00 40 00 00  tres.bin.....@..
00F09000  75 6E 6F 2E 62 69 6E 00 00 00 00 00 00 10 00 00  uno.bin......►..
00F09010  64 6F 73 2E 62 69 6E 00 00 00 00 00 00 00 05 00  dos.bin.......♣.
00F09020  60 6A 04 68 00 30 00 00 68 00 10 00 00 68 00 00  `j♦h.0..h.►..h..
00F09030  E4 01 FF 15 00 F0 8F 00 85 C0 6A 04 68 00 30 00  ä0 §.-Ä.└j♦h.0.
00F09040  00 68 00 00 05 00 68 00 00 E8 01 FF 15 00 F0 8F  .h..♣.h..þ0 §.-Ä
```

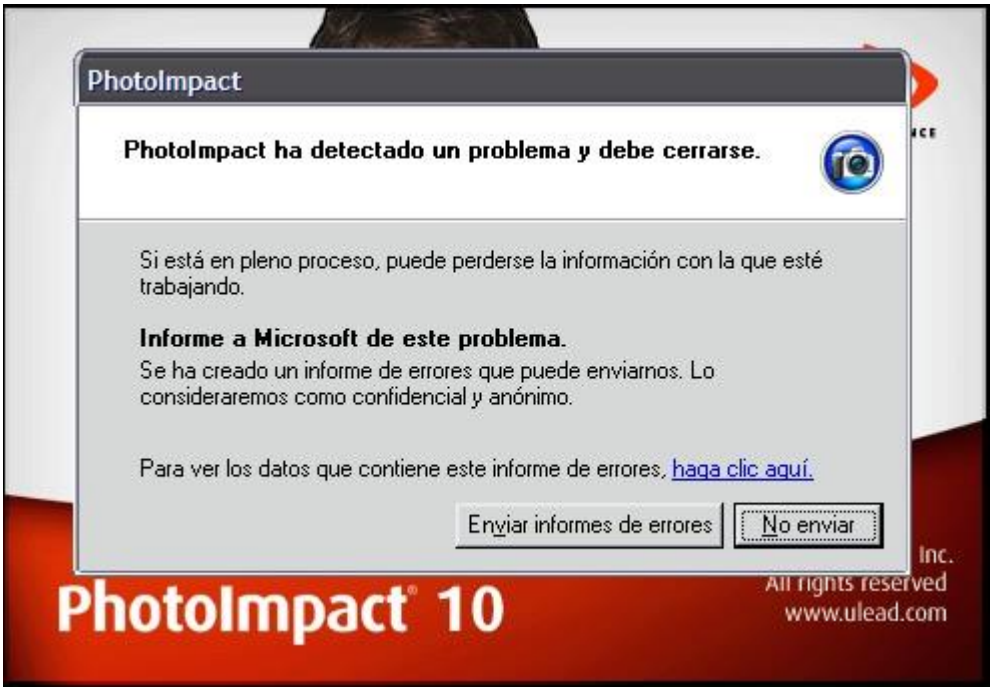
Los debéis poner ahí. Y una cosa más que me tiene mosqueado, cuando reparamos la ***falsa_iat*** para importar las funciones necesarias le dijimos que la pusiera a partir de ***:8FF000***, pues bien si vamos allí:

```
Address   Hex dump                                          ASCII
008FF000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF090  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF100  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF110  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF120  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF130  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

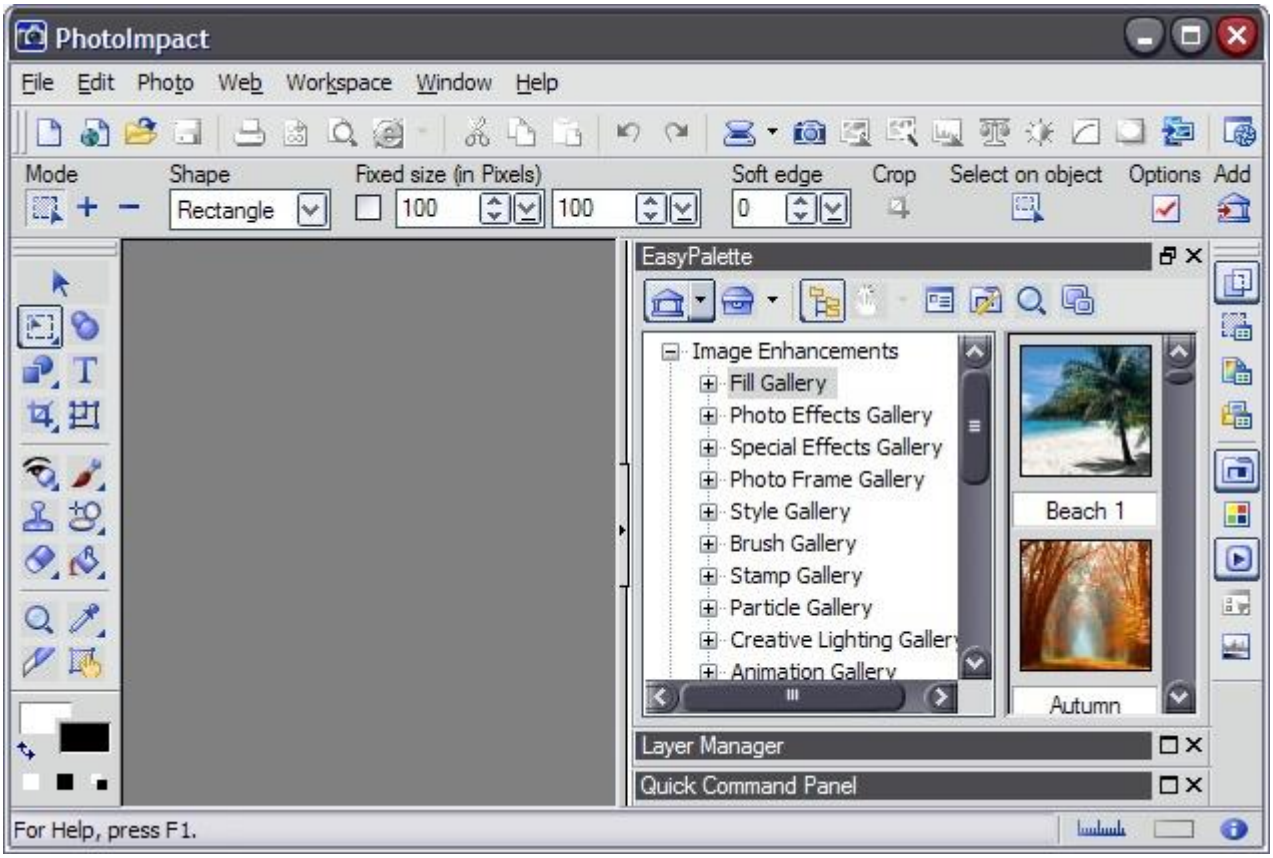
No ha puesto nada, pensé que nos habíamos quedado sin permiso de escritura pero no es así, lo que hice fué ponerlos a mano y salvar el archivo después, primero lo dejé así como en la imagen:

```
Address   Hex dump                                          ASCII
008FF000  72 AC E5 77 76 B4 E5 77 4E AB E5 77 00 00 00 00  r¼ówv┤ówN½ów....
008FF010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF090  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF0F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF100  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF110  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF120  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
008FF130  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Esto me tiene preocupado porque quizá genere que le programa no corra en otra máquina, de esto no estoy seguro porque aún no lo he podido comprobar. Pero antes de preocuparnos si funcionará en otra máquina tenemos que mirar si funciona en la nuestra, así que todos a ejecutar el archivo...




El muy mamón saca el baner pero peta al momento, cambiémosle el nombre de ***dumped_.exe*** a ***Iedit.exe***:




Chavales esto FUNCIONAAAAAA...!!!

Toquetead todas las funciones que queráis, funciona perfecto, al final cayó, ha costado pero cayó, jeje.. como siempre volvemos a demostrar que un poquito de perseverancia y un par de ideas despendolás fulminan cualquier protección comercial de este tipo, la verdad que la idea de la iat es buenísima, sí señor pero no ha sido suficiente para minar nuestra nimia paciencia ;XDDD

Espero que os haya gustado el tuto tanto como a mi escribirlo, con esto pretendo ayudar a aquellos que empiezan y necesitan un pequeño empujón para arrancar, es por eso que me paro a explicar pequeños detalles que parecen triviales y que se dan por hechos, yo como siempre me pongo en la piel del que empieza porque me ha tocado vivirlo y se lo que es y de verdad que toda ayuda ***ES POCA***..!!

Venga chic@s que ya queda menos, ganaremos la carrera...

Deciros que he estado probando las funciones por encima y no vi ninguna limitación, salvé una imagen y no insertó ningún logo ni nada por lo que intuyo que la única limitación era la temporal y cayó con el packer ;XDDD

Y al fin terminamos, para cualquier sugerencia, duda, pregunta y por supuesto crítica pueden escribirme a cualquiera de estas dos direcciones, como siempre decir que todos los mensajes serán contestados:

syxe00@yahoo.es
syxe00@homail.com

Chao, hasta la próxima... Si da problemas en otras máquinas por favor hacérmelo saber, os lo agradeceré.

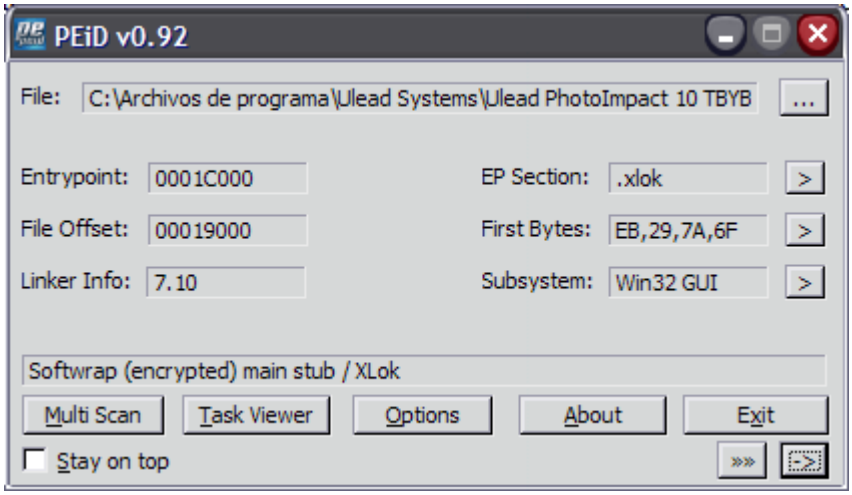
***ANEXO***: No sé como me ha dado por adelantar la fecha del reloj y mirad que me encontré:

Este no es como ***Armadillo*** o ***ASProtect*** que al desempaquetarlo ya no expira, o sea que tenemos más trabajo, por lo que he visto no se puede tracear muy bien que se diga, además la librería ***Softwrap.dll*** está comprimida también con el mismo packer e imagino que es la que lleva la limitación temporal.

Cargo el programa en ***OllyDbg*** pongo en el ***CommandBar*** un ***BP MessageBoxA*** y lo dejo correr pulsando ***F9***, me avisa de que el módulo ***Register*** tiene el ***EP*** fuera de la seción ***.code*** ¿empaquetado también?,jeje.. fijo que si, bueno lo interesante es que antes del mensaje ***Olly*** rompe, veamos en el stack que hay:

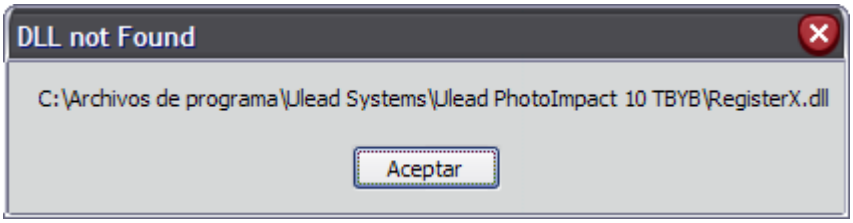
```
0012F380  01E122EF  CALL to MessageBoxA from xsystem.01E122E9
0012F384  00000000  hOwner = NULL
0012F388  0012F4C8  Text = "C:\Archivos de programa\Ulead Systems\Ulead PhotoImpact 10 TBYB\RegisterX.dll"
0012F38C  01E14238  Title = "DLL not Found"
0012F390  00000000  Style = MB_OK|MB_APPLMODAL
0012F394  77F69005  ntdll.77F69005
0012F398  77F5DA88  ntdll.77F5DA88
0012F39C  00000000
```

Pues el culpable parece ser ***RegisterX.dll***, vaya nombre no? no va a pasar desapercibido no! pasémosle el ***PEiD***:



Pues ahí lo ven, esto es una pesadilla, debemos atacar la comprobación temporal dentro de un archivo ***.dll*** empaquetado con el mismo ***Softwrap*** que acabamos de vencer, veamos que se puede hacer ¡¡!!

Pues no es este mensaje, pulso ***Ctrl+F9*** para llegar al ***ret*** y aparee en pantalla:




En mi humilde opinión esto es un ***anti-traceo*** como una casa, supero el ***ret*** y mirad que vemos:

```
0003Dbg - ledit.exe - [CPU - main thread, module xsystem]
File View Debug Plugins Options Window Help

01E122BB   83C8 FF          OR EAX,FFFFFFFF
01E122BE   E9 58020000      JMP xsystem.01E1251B
01E122C3   8B4D 08          MOV ECX,DWORD PTR SS:[EBP+8]
01E122C6   51               PUSH ECX
01E122C7   FF15 2430E101    CALL DWORD PTR DS:[<&KERNEL32.LoadLibra kernel32.LoadLibraryA
01E122CD   8985 F4FEFFFF    MOV DWORD PTR SS:[EBP-10C],EAX
01E122D3   83BD F4FEFFFF    CMP DWORD PTR SS:[EBP-10C],0
01E122DA   75 1B            JNZ SHORT xsystem.01E122F7
01E122DC   6A 00            PUSH 0
01E122DE   68 3842E101      PUSH xsystem.01E14238          ASCII "DLL not Found"
01E122E3   8B55 08          MOV EDX,DWORD PTR SS:[EBP+8]
01E122E6   52               PUSH EDX
01E122E7   6A 00            PUSH 0
01E122E9   FF15 7C30E101    CALL DWORD PTR DS:[<&USER32.MessageBoxA USER32.MessageBoxA
01E122EF   83C8 FF          OR EAX,FFFFFFFF
01E122F2   E9 24020000      JMP xsystem.01E1251B
01E122F7   68 4842E101      PUSH xsystem.01E14248          ASCII "IsRegistered"
01E122FC   8B85 F4FEFFFF    MOV EAX,DWORD PTR SS:[EBP-10C]
01E12302   50               PUSH EAX
01E12303   FF15 2030E101    CALL DWORD PTR DS:[<&KERNEL32.GetProcAd kernel32.GetProcAddress
01E12309   8985 E8FEFFFF    MOV DWORD PTR SS:[EBP-118],EAX
01E1230F   83BD E8FEFFFF    CMP DWORD PTR SS:[EBP-118],0
01E12316   75 1E            JNZ SHORT xsystem.01E12336
01E12318   6A 00            PUSH 0
01E1231A   8B4D 08          MOV ECX,DWORD PTR SS:[EBP+8]
01E1231D   51               PUSH ECX
01E1231E   68 5842E101      PUSH xsystem.01E14258          ASCII "Can not find register function"
01E12323   6A 00            PUSH 0
01E12325   FF15 7C30E101    CALL DWORD PTR DS:[<&USER32.MessageBoxA USER32.MessageBoxA
01E1232B   FF15 1C30E101    CALL DWORD PTR DS:[<&KERNEL32.GetLastEr ntdll.RtlGetLastWin32Error
01E12331   E9 E5010000      JMP xsystem.01E1251B
01E12336   C785 E0FEFFFF    MOV DWORD PTR SS:[EBP-120],0
01E12340   EB 0F            JMP SHORT xsystem.01E12351
01E12342   8B95 E0FEFFFF    MOV EDX,DWORD PTR SS:[EBP-120]
01E12348   83C2 01          ADD EDX,1
01E1234B   8995 E0FEFFFF    MOV DWORD PTR SS:[EBP-120],EDX
01E12351   83BD E0FEFFFF    CMP DWORD PTR SS:[EBP-120],0A
```

Está claro que debíamos haber saltado en ***:1E122DA*** y por alguna razón no saltó, si hubiera saltado hubiésemos llegado a ***:1E122F7*** donde aparee un interesante ***IsRegistered*** que no necesita explicación, si conseguimos desempaquetar la ***dll*** esto es pan comido pero eso tampoco será fácil, pero ojo que el código anterior está en ***xsystem.dll*** que no está empaquetada afortunadamente, bouuuuuff..!!

pero no nos alegremos mucho todavía porque si modificamos ***xsystem.dll*** resulta que ***u32cfg.dll*** que por lo visto no tiene nada mejor que hacer nos saca un mensaje de error tipo ***CRC32*** quejándose de los archivos de licencia o algo así, si seguimos la llamada acabaremos retornando a:

```
:00531B20   8B8424 2C0200     MOV EAX,DWORD PTR SS:[ESP+22C]
:00531B27   85C0              TEST EAX,EAX
:00531B29   75 0F             JNZ SHORT Iedit.00531B3A
:00531B2B   6A 00             PUSH 0
:00531B2D   68 E8030000       PUSH 3E8
:00531B32   E8 0BB03900       CALL Iedit.008CCB42     ;Esta es la llamada maldita..!!
:00531B37   83C4 08           ADD ESP,8               ;Aquí salimos..!!
:00531B3A   8B4424 10         MOV EAX,DWORD PTR SS:[ESP+10]
:00531B3E   5E                POP ESI
:00531B3F   5D                POP EBP
:00531B40   83F8 03           CMP EAX,3
:00531B43   5B                POP EBX
```

bueno pues estamos otra vez dentro de ***Iedit.exe***, parece que aquí todos los archivos están entrelazados entre sí, ojo con poner ***BPX*** o con intentar superar un call con ***F8*** porque se produce una excepción y nos tira fuera, la manera que yo utilizo para tracear por aquí es con ***HE*** de corto alcance ya que los pongo y los quito continuamente para ser detectado, es engorroso pero va funcionando, lo que hice fue poner un ***BPHE*** al inicio de la zona para parar allí y tracear como os he comentado, así ver un poco más como trabaja

```
:00531AB7   FFD5              CALL EBP                        ;Aquí pongo el HE (en realidad lo puse en :531A36)
:00531AB9   83C4 14           ADD ESP,14
:00531ABC   85C0              TEST EAX,EAX
:00531ABE   74 45             JE SHORT Iedit.00531B05
:00531AC0   8B5424 10         MOV EDX,DWORD PTR SS:[ESP+10]
:00531ACE   FFD6              CALL ESI
:00531AD0   8D8C24 2C010000   LEA ECX,DWORD PTR SS:[ESP+12C]
:00531AD7   8D5424 28         LEA EDX,DWORD PTR SS:[ESP+28]
:00531ADB   51                PUSH ECX
:00531ADC   52                PUSH EDX
:00531ADD   FF15 2C63EB00     CALL DWORD PTR DS:[EB632C]          ; Iedit.00EB8EA7
:00531AE3   83C4 14           ADD ESP,14
:00531AE6   85C0              TEST EAX,EAX
:00531AE8   75 13             JNZ SHORT Iedit.00531AFD
:00531AEA   8B4424 10         MOV EAX,DWORD PTR SS:[ESP+10]
:00531AEE   83F8 01           CMP EAX,1
:00531AF1   74 12             JE SHORT Iedit.00531B05
:00531AF3   83F8 03           CMP EAX,3
:00531AF6   74 0D             JE SHORT Iedit.00531B05
:00531AF8   83F8 02           CMP EAX,2
:00531AFB   74 08             JE SHORT Iedit.00531B05
:00531AFD   C74424 10 02000>  MOV DWORD PTR SS:[ESP+10],2
:00531B05   8B8424 2C020000   MOV EAX,DWORD PTR SS:[ESP+22C]
:00531B0C   85C0              TEST EAX,EAX
:00531B0E   75 2A             JNZ SHORT Iedit.00531B3A        ;Evitamos el call
:00531B10   837C24 10 02      CMP DWORD PTR SS:[ESP+10],2
:00531B15   75 23             JNZ SHORT Iedit.00531B3A        ;Evitamos el call
:00531B17   6A 00             PUSH 0
:00531B19   68 E9030000       PUSH 3E9
:00531B1E   EB 12             JMP SHORT Iedit.00531B32
:00531B20   B8 03000000       MOV EAX,3
:00531B25   90                NOP
:00531B26   90                NOP
:00531B27   85C0              TEST EAX,EAX
:00531B29   EB 0F             JMP SHORT Iedit.00531B3A
:00531B2B   6A 00             PUSH 0
:00531B2D   68 E8030000       PUSH 3E8
:00531B32   E8 0BB03900       CALL Iedit.008CCB42             ;Este es el Call_Restrictor..!!
:00531B37   83C4 08           ADD ESP,8
:00531B3A   8B4424 10         MOV EAX,DWORD PTR SS:[ESP+10]
```

Estamos en la zona donde sin duda se está chequeando el estado de usuario, los calls iniciales son del tipo ***xCipher***, otros del tipo ***xDeCipher*** que lo que hacen es lógicamente encriptar y desencriptar datos, datos que imagino ha sacado de los archivos de licencia que tiene escondidos por nuestro ***HDD***, aunque a nosotros esto tampoco nos incumbe porque pasamos un rato largo de sus complejos algoritmos, después de verificar en sucesivos calls vemos un par de saltos que evitan el ***call_Restrictor*** que nos saca todos los mensajes y nos tira a la calle, si probamos a jumpear uno ***Olly*** se cierra sin más ***¿?*** Parece que ha detectado el cambio, entonces hagamos otra cosa modifiquemos también el flag ***[ESP+10]*** que controla el salto, pues quizá lo vuelva a comprobar más adelante y si ve allí un 2, jeje.. sabe que le hemos cambiado el salto, primero comentar que mientras traceamos a modo de ***HE*** esta zona (***BPX*** no se os ocurra) vemos en el ***stack*** esta cadena:

```
0012F8D8  0012F908  ASCII "{D40ED277-646E-5F82-3418-95BB91E4D524}"
0012F8DC  0012FA0C  ASCII "{D40ED277-646E-5F82-3418-95BB91E4D524}"
0012F8E0  0012FA0C  ASCII "{D40ED277-646E-5F82-3418-95BB91E4D524}"
0012F8E4  00000002
0012F8E8  01E10000  ASCII "MZE"
0012F8EC  00992ED0  ASCII "P "
```

La pongo por si hay alguien a quien le interese estudiar más a fondo esta rutina, a mi por ejemplo me da igual pero a lo mejor hay gente que no le da igual y quiere investigar más, allá cada uno es libre.

Lo dicho, miraremos de modificar el salto y el valor de ***[ESP+10]***, pero primeramente tenemos que ver cuanto vale, ***Follow in Dump*** y vemos que representa y que valor tiene en la ventana ***Dump***:

```
Address   Hex dump                                          ASCII
0012F8FC  02 00 00 00 00 00 ED 01 00 10 ED 01 7B 46 34 34  ☻.....ÿ0.►ÿ0{F44
0012F90C  32 44 32 38 33 2D 36 34 42 41 2D 35 46 38 32 2D  2D283-64BA-5F82-
0012F91C  37 38 31 38 2D 35 39 42 42 39 31 33 30 44 39 39  7818-59BB9130D99
```

Pues representa a ***[12F8FC]*** y tiene valor ***2***, claro si cambiamos el salto y no este valor es fácil darse cuenta del cambiazo pero ¿y si cambiamos el salto y este valor? lo probamos:

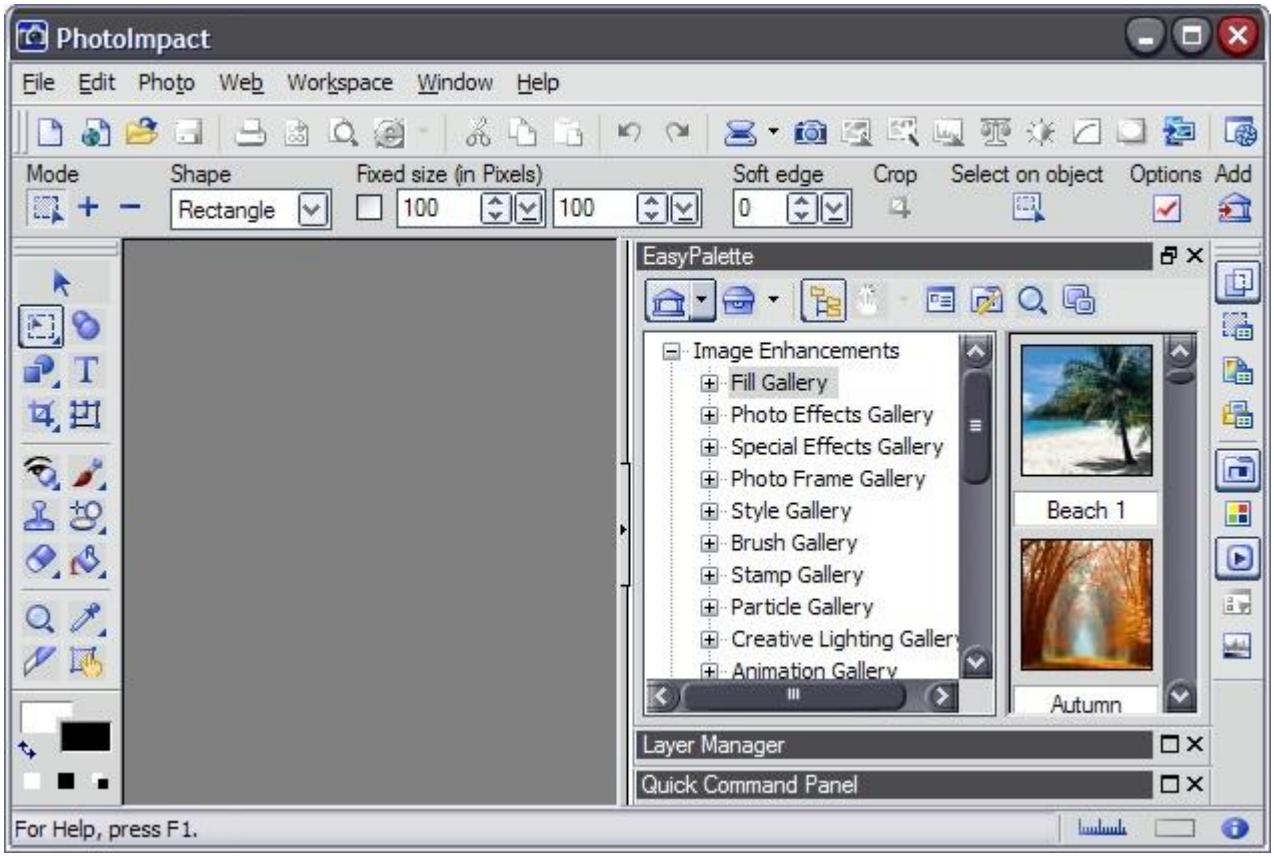
```
Address   Hex dump                                          ASCII
0012F8FC  01 00 00 00 00 00 ED 01 00 10 ED 01 7B 46 34 34  ☺.....ÿ0.►ÿ0{F44
0012F90C  32 44 32 38 33 2D 36 34 42 41 2D 35 46 38 32 2D  2D283-64BA-5F82-
0012F91C  37 38 31 38 2D 35 39 42 42 39 31 33 30 44 39 39  7818-59BB9130D99
```

Ahora cambiamos también el salto:

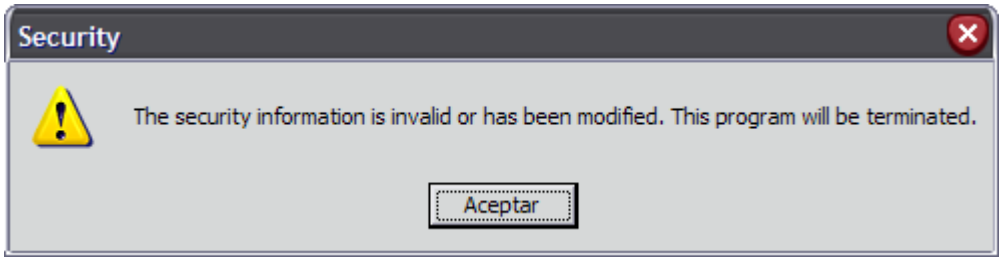
```
0003Dbg - ledit.exe - [CPU - main thread, module ledit]
File View Debug Plugins Options Window Help

00531B0C   85C0          TEST EAX,EAX
00531B0E   75 2A         JNZ SHORT ledit.00531B3A
00531B10   837C24 10 02  CMP DWORD PTR SS:[ESP+10],2
00531B15   EB 23         JMP SHORT ledit.00531B3A
00531B17   6A 00         PUSH 0
00531B19   68 E9030000   PUSH 3E9
00531B1E   EB 12         JMP SHORT ledit.00531B32
00531B20   B8 03000000   MOV EAX,3
00531B25   90            NOP
00531B26   90            NOP
00531B27   85C0          TEST EAX,EAX
00531B29   EB 0F         JMP SHORT ledit.00531B3A
00531B2B   6A 00         PUSH 0
00531B2D   68 E8030000   PUSH 3E8
00531B32   E8 0BB03900   CALL ledit.008CCB42
00531B37   83C4 08       ADD ESP,8
00531B3A   8B4424 10     MOV EAX,DWORD PTR SS:[ESP+10]
00531B3E   5E            POP ESI
```

Ahora pasará por encima del ***call_Restrictor***, pulsamos ***F9*** y....




SIIIIIIIIIIIIIIIIII... vuelve a vivir, jeje..




Este es el mensaje que mostraba ***u32cfg.dll*** que se me pasó mostrarlo antes, pero más vale tarde que nunca o al menos eso dicen no?

Ahora la pregunta es ¿detectará un cambio permanente? Lo probaremos, cambiamos:

```
:00531B10   837C24 10 02      CMP DWORD PTR SS:[ESP+10],2
:00531B15   75 23             JNZ SHORT Iedit.00531B3A
```

***POR***:

```
:00531B10   C64424 10 01      MOV BYTE PTR SS:[ESP+10],1
:00531B15   EB 23             JMP SHORT Iedit.00531B3A
```

Y salvamos los cambios desde el propio ***Olly***, ejecutamos el archivo y... ***FUNCIONA!***

Echemos el reloj para atrás como 2 ó 3 años y... ***FUNCIONA!***

Y ahora sí ***CREO*** que hemos terminado ya que el programa corre independientemente del estado de los archivos de licencias y todo eso, si a alguien le interesa tengo un ****.log*** de los indicadores que tiene en el registro y en el ***HDD*** en mi máquina, en la tuya no será muy diferente, si te interesa pues me mandas un mail que te los pasaré encantado, y ya está bien por hoy otro día más y mejor.

Alguna duda, sugerencia, pregunta o crítica ya sabéis, escribidme a:

syxe00@yahoo.es
syxe00@hotmail.com

Por supuesto todos los mensajes serán contestados me comprometo, chao..!!